

Stack Data Structure

1)

QUESTIONS 10 / 10 completed

Stack using Array

STACK USING ARRAY

"A web browser maintains a history of recently visited pages. Write a C program to implement a stack using an array and perform Push, Pop, Peek, and Display operations while handling overflow and underflow conditions. Test Case 1"

PROGRAM EDITOR

```
1 #include <stdio.h>
2 #define MAX 5
3
4 int stack[MAX];
5 int top = -1;
6
7 // Push operation
8 void push(int value) {
9     if (top == MAX - 1) {
10         printf("Stack Overflow\n");
11         return;
12     }
13     stack[++top] = value;
14     printf("%d pushed into stack\n", value);
15 }
16
17 // Pop operation
18 void pop() {
19     if (top == -1) {
20         printf("Stack Underflow\n");
21         return;
22     }
23     printf("%d popped from stack\n", stack[top]);
24     top--;
25 }
```

Run Save

INPUT

Stack Size = 5
Push: 10, 20, 30
Display

2)

QUESTIONS 10 / 10 completed

Stack using Linked

STACK USING LINKED LIST

"A document editor maintains recently performed actions to support the Undo feature. Implement a stack using a linked list and perform push, pop, and display operations. Test Case 1"

Input

PROGRAM EDITOR

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 struct Node *top = NULL;
10
11 // Push operation
12 void push(int value) {
13     struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
14     newNode->data = value;
15     newNode->next = top;
16     top = newNode;
17     printf("%d pushed into stack\n", value);
18 }
19
20 // Pop operation
21 void pop() {
```

Run Save

INPUT

Push 100, 200, 300
Display

3)

QUESTIONS 10 / 10 completed

Stack -Postfix Expre

STACK -POSTFIX EXPRESSION

"Write a C program that uses a stack to evaluate a postfix expression and display the result. Postfix expressions are commonly used in compilers because they eliminate the need for parentheses. Input:

PROGRAM EDITOR

C

Run

Save

```

1 #include <stdio.h>
2 #include <ctype.h>
3
4 #define MAX 100
5
6 int stack[MAX];
7 int top = -1;
8
9 void push(int x)
10 {
11     stack[++top] = x;
12 }
13
14 int pop()
15 {
16     return stack[top--];
17 }
18
19 int main()
20 {
21     char postfix[MAX];

```

INPUT

23*5+

4)

QUESTIONS 10 / 10 completed

Stack -Infix to Post'

STACK -INFIX TO POSTFIX

"Arithmetic expressions entered by programmers are generally in infix form. Before evaluation, many systems convert them into postfix notation. Write a C program using a stack to convert an infix expression into its equivalent postfix. Input:

PROGRAM EDITOR

C

Run

Save

```

1 #include <stdio.h>
2 #include <ctype.h>
3 #include <string.h>
4
5 #define MAX 100
6
7 char stack[MAX];
8 int top = -1;
9
10 void push(char ch)
11 {
12     stack[++top] = ch;
13 }
14
15 char pop()
16 {
17     return stack[top--];
18 }
19
20 char peek()
21 {

```

INPUT

A+B*C

5)

QUESTIONS 10 / 10 completed

Stack- Balanced Pa

STACK-BALANCED PARENTHESES

"Compilers must verify whether all opening parentheses have corresponding closing parentheses. Write a C program using a stack to check whether an expression contains balanced parentheses. Input:

PROGRAM EDITOR

C

Run

Save

```

1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAX 100
5
6 char stack[MAX];
7 int top = -1;
8
9 void push(char ch)
10 {
11     stack[++top] = ch;
12 }
13
14 char pop()
15 {
16     return stack[top--];
17 }
18
19 int main()
20 {
21     char exp[MAX];

```

INPUT

(A+B)*(C-D)

6)

QUESTIONS 10 / 10 completed

Stack - Reverse Stri

STACK - REVERSE STRING

"A text-processing application requires reversing user-entered strings. Write a C program that uses a stack to reverse a given string.
Input

DATASTRUCTURE

Output

PROGRAM EDITOR

C

Run Save

```

1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAX 100
5
6 char stack[MAX];
7 int top = -1;
8
9 void push(char ch)
10 {
11     stack[++top] = ch;
12 }
13
14 char pop()
15 {
16     return stack[top--];
17 }
18
19 int main()
20 {
21     char str[MAX];

```

INPUT

DATASTRUCTURE

7)

QUESTIONS 10 / 10 completed

Stack - Decimal to f

STACK - DECIMAL TO BINARY

"Computer systems internally represent numbers in binary format. Write a C program that uses a stack to convert a decimal number into its binary equivalent.
Input

13

PROGRAM EDITOR

C

Run Save

```

1 #include <stdio.h>
2
3 #define MAX 100
4
5 int stack[MAX];
6 int top = -1;
7
8 void push(int x)
9 {
10     stack[++top] = x;
11 }
12
13 int pop()
14 {
15     return stack[top--];
16 }
17
18 int main()
19 {
20     int n;
21

```

OUTPUT

INPUT

13

8)

QUESTIONS 10 / 10 completed

Tower of Hanoi Prc

TOWER OF HANOI PROBLEM

"The Tower of Hanoi is a classic problem demonstrating recursion and stack behavior. Write a C program to solve the Tower of Hanoi problem and display the sequence of moves required to transfer all disks.
Input

PROGRAM EDITOR

C

Run Save

```

1 #include <stdio.h>
2
3 void towerOfHanoi(int n, char source, char auxiliary, char destination)
4 {
5     if (n == 1)
6     {
7         printf("Move Disk 1 from %c to %c\n", source, destination);
8         return;
9     }
10
11     towerOfHanoi(n - 1, source, destination, auxiliary);
12     printf("Move Disk %d from %c to %c\n", n, source, destination);
13     towerOfHanoi(n - 1, auxiliary, source, destination);
14 }
15
16 int main()
17 {
18     int n;
19

```

OUTPUT

INPUT

3

9)

QUESTIONS 10 / 10 completed

Stack - Browser His

STACK - BROWSER HISTORY

"A web browser stores visited web pages in a stack to enable navigation to previously visited pages. Write a C program that simulates browser history using a stack and supports back navigation. Input

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAX 10
5 #define SIZE 50
6
7 char stack[MAX][SIZE];
8 int top = -1;
9
10 void push(char page[])
11 {
12     if(top < MAX - 1)
13     {
14         top++;
15         strcpy(stack[top], page);
16     }
17 }
18
19 char* pop()
20 {
21     if(top >= 0)
```

INPUT

Visit: Google
Visit: YouTube
Visit: Wikipedia
Back

OUTPUT

10)

QUESTIONS 10 / 10 completed

Stack- Undo Opera

STACK- UNDO OPERATION

"A text editor provides an Undo feature that allows users to revert the most recent action. Write a C program using a stack to simulate an Undo operation for text editing commands Input

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAX 100
5
6 char stack[MAX];
7 int top = -1;
8
9 void push(char ch)
10 {
11     stack[++top] = ch;
12 }
13
14 char pop()
15 {
16     if(top == -1)
17         return '\0';
18     return stack[top--];
19 }
20
21 int main()
```

INPUT

Insert A
Insert B
Insert C
Undo